Nobody Left Behind

Why each feature is needed, how many screens to build, and how many roles you need

Stack: Next.js + Tailwind (Vercel free), Supabase (Postgres/PostGIS, Auth, Realtime, Edge Functions), MapLibre, Telegram bot, Open-Meteo. Everything is on free tiers.

5	12	17	8
roles in the database (4 active + 1 read-only)	routes (pages)	distinct screens / views including tabs and drawer	pages for a minimum viable demo

The short version

The problem statement asks for two linked jobs: **(a) reach people that warnings miss** and **(b) send scarce resources to the neediest first**, while staying honest about bad data and network failure. Every screen and role below exists because one of those jobs has a real person doing it. Residents never log in; they are reached through Telegram, SMS or voice, so they cost you zero screens but are the most important users.

Roles and screens are counted three ways below: the full build, a realistic hackathon build, and the absolute minimum. Pick the one that matches your team size and days.

1. Roles: how many and why

There are **5 database roles** (stored in `profiles.role`) plus **residents**, who are not users of the app at all. Each role maps to a different real-world job, which is also what makes your Row Level Security (privacy) story credible.

Role	Why it is needed	Pages	Data access (RLS)	Priority
Coordinator	Makes the final call. The problem says the system recommends and a human decides, so someone must accept, override and be accountable.	/command, /dispatch, /simulate, /quality, /intake, /admin/demo	Everything, including exact locations of sensitive households and the audit log.	Essential
Field volunteer	The 'last mile'. People with no phone or no smartphone can only be reached by a person walking or calling. Needs a mobile, offline-capable view.	/field	Only the tasks and people assigned to them.	Essential
Community registrar	Finds people nobody registered (lady health worker, mosque or school committee, councillor). Without this role the vulnerable list stays empty.	/register	Only the households they registered.	Essential
Shelter manager	Capacity and special needs (ramps, women-only areas, medical) can only be known on site. Closes the loop from 'rescued' to 'housed'.	/shelter	Their own shelter, arrivals and supplies.	Nice to have
Viewer / judge	Read-only demo access so judges can click around without breaking anything and without seeing sensitive data.	/command (read-only), /quality	Aggregated, anonymised data only.	Optional
Resident (no login)	The person being protected. Reached through Telegram, SMS or an IVR call; replies 1/2/3 to check in. Silence is treated as a signal.	Telegram bot + /resident-sim	Their own messages only.	Essential (but not an app role)

Minimum viable roles: 3 (coordinator, volunteer, registrar) plus residents via the simulator. Shelter manager and viewer can be added later without any redesign because RLS rules are per role.

2. Screens: how many and what each is for

12 routes become **17 distinct screens** once you count the case drawer, the four views inside the volunteer app, and the three tabs on the quality page.

#	Route / view	Role	What it shows	Why it is needed
1	/login	All	Role-based sign-in with one-click demo logins.	Without role routing the privacy model is not demonstrable, and you waste demo time typing passwords.
2	/command	Coordinator	Map, priority queue, resource board, hazard clock.	This is the answer to 'who gets help first'. Every other page feeds it.
3	Case drawer (/case/[id])	Coordinator	Score breakdown, confidence, check-in history, accept / override / mark safe.	Judges must see the reason behind each ranking, and overrides need a required reason for accountability.
4	/dispatch	Coordinator	Proposed resource assignments, ETA, trade-off notes, pin and recompute.	Proves resources go to the neediest first and shows the competing-priorities trade-off explicitly.
5	/simulate	Coordinator	What-if toggles: rainfall, network down, unreliable reports, road closed, boat broken.	The ground rules demand behaviour under bad data and failure. This screen shows it in seconds.
6	/field: Tasks	Volunteer	Next task card, upcoming list, online/offline pill.	Gives the last-mile volunteer one clear instruction.
7	/field: Task detail	Volunteer	Mini-map, call button, Reached / Can't reach / Need backup.	One-tap status keeps the command centre accurate even on a bad network.
8	/field: Buddy check-ins	Volunteer	Assigned people with call and outcome selector.	Delivers the buddy-volunteer idea for people with no phone.
9	/field: Sync status	Volunteer	'3 updates waiting' outbox view.	Makes offline mode visible, which is a core requirement.
10	/resident-sim	Demo / any	Phone mockup (Telegram, SMS, voice call) plus router decision panel.	Residents have no app, so this screen is how you show channel routing and silence escalation.
11	/register	Registrar	Form, 'my registrations' list, SMS-style quick entry.	Solves 'how do you find people nobody told you about'.
12	/shelter	Shelter manager	Capacity bar, arrivals, special-needs flags, overflow suggestion.	Completes the pipeline: rescue to shelter.
13	/quality: Data quality	Coordinator	Confidence map, count of inferred scores, pessimistic top-20.	Shows missing data is marked 'unknown, high exposure', never 'safe'.
14	/quality: Fairness	Coordinator	Rank vs report-rate chart, gap flag, one-click correction.	Quiet areas report less. This proves they are not ranked lower for it.
15	/quality: Audit	Coordinator	Searchable override log with before/after.	Evidence of human control and accountability.
16	/intake	Coordinator	Raw message (Roman Urdu) beside parsed fields and confidence.	Handles messy, duplicate, wrong reports; low confidence goes to a human.
17	/admin/demo	Coordinator	Reset, seed, play storm, jump in time, break network, close road.	Makes the 5-minute demo repeatable and protects you from one bug.

Three build sizes

Size	Screens	Roles	What you get
Minimum viable demo	8 pages: login, command (with case drawer), dispatch, resident-sim, field (4 views), register, simulate, admin/demo	3 roles + residents	Core loop works: score, warn, check in, escalate, dispatch, override, offline.
Realistic hackathon build	12 routes / 15 screens: add quality (data and fairness), shelter, intake	4 roles + residents	Adds the robustness and ethics evidence judges look for.
Full build	12 routes / 17 screens as listed above	5 roles + residents	Everything in this document.

Role-to-screen map

Screen	Coordinator	Volunteer	Registrar	Shelter mgr	Viewer
/command, case drawer	X				view
/dispatch	X				
/simulate	X				
/field (4 views)		X			
/register			X		
/shelter				X	
/quality (3 tabs)	X				aggregates
/intake	X				
/admin/demo	X				
/resident-sim	X	X	X	X	X

3. Features: what each is and why it is needed

Each row is tied back to the problem statement. If you cannot explain why a feature exists in one sentence, it is a candidate to cut.

Layer 1: Know who is vulnerable

Feature	Why it is needed	Where
Risk score = exposure x vulnerability x time pressure	A single, explainable number lets you rank thousands of people. Multiplying means a safe, healthy person on high ground scores low, and a frail person in a flood zone scores high.	Command, drawer
Confidence value on every score	Data is always incomplete. Showing confidence stops the system from looking more certain than it is.	Drawer, quality
Unknown = 'high exposure, welfare check'	The dangerous failure is treating missing data as 'safe'. This rule makes silence trigger action.	Map hatch, quality
Community registry (registrars)	People who are alone, elderly or without phones are not in any database. Trusted local people are the only realistic source.	/register
Self and neighbour registration (SMS or Telegram)	Some people can register themselves, and neighbours often know who is alone.	Telegram, /register
Area-level inference	Where household data is missing you still need an estimate, with a wider uncertainty band.	Quality
Silence as a signal	A person who does not reply is the one most likely to be in trouble.	Check-ins

Layer 2: Reach people the alerts miss

Feature	Why it is needed	Where
Channel router (smartphone, SMS, voice, buddy)	One alert on one channel misses most vulnerable people. Matching the channel to the person is the central idea of the problem.	/resident-sim
Telegram bot (real) + SMS/voice simulator	Telegram is free and instant. SMS and voice to Pakistani numbers are awkward on free tiers, so simulate them behind the same function.	/resident-sim
Buddy volunteer last mile	A person with no phone can only be warned by a human.	/field
Two-way check-in (reply 1/2/3)	A one-way alert tells you nothing about whether it worked.	Telegram, simulator
Automatic escalation (reminder, call, buddy, dispatch)	Turns silence into action without a coordinator watching every person.	Check-ins, /field
Vulnerability-adjusted warning lead time	The elderly and disabled need more time to move, so they are warned earlier than everyone else. This is your most distinctive idea.	Hazard clock
Local languages (Urdu, Sindhi, Roman Urdu)	A warning people cannot read or understand is not a warning.	Router, intake
Heatwave mode	Same system, different rules. Shows the design generalises beyond floods.	Rules config

Layer 3: Send help to the neediest first

Feature	Why it is needed	Where
Priority tiers (life threat first)	Competing priorities need a rule everyone can predict. Life-threatening cases must never be outranked by a high score elsewhere.	/command
Transparent triage score inside each tier	Responders will only trust a ranking they can read and adjust.	Drawer
Optimizer (Hungarian matching on a road graph)	Matches limited boats, ambulances and teams to cases by harm reduced against travel time. Simple, fast and explainable.	/dispatch

Feature	Why it is needed	Where
Plain-language reason per assignment	Example: 'Boat 2 to Mrs. Khan, cannot self-evacuate, oxygen-dependent, ranks 2nd of 41.'	/dispatch
Trade-off explainer	Every assignment delays someone else. Showing that openly is what 'competing priorities' means.	/dispatch
Human override with required reason	The system recommends; a person decides and is accountable.	Drawer, audit
Shelter matching	Rescue is pointless without a suitable place to go (ramp, women-only, medical).	/shelter

Layer 4: Act early and cope with failure

Feature	Why it is needed	Where
Forecast to time-to-impact countdown	Acting before the water arrives is what saves the people who cannot self-evacuate.	Top bar
Pre-positioning recommendations	Boats and medicine must be where roads will be cut off, before they are cut off.	/dispatch
Staged alerts (advisory, warning, evacuation)	Gives different groups the right message at the right time instead of one panic alert.	Hazard clock
Report intake parsing + review queue	Real reports are messy, in mixed languages, and sometimes wrong.	/intake
Deduplication and contradiction flags	Ten reports of one flooded street must not become ten emergencies.	/intake
Offline field app (PWA + outbox)	Networks and power fail in a disaster. Volunteers must keep working and sync later.	/field
SMS fallback and printed priority list	Works on basic phones and weak networks.	Router, /admin
Early evacuation of power-dependent patients	Oxygen and dialysis patients are at risk the moment the grid fails, before the water arrives.	Rules
What-if simulator	Proves behaviour under bad data and network failure instead of just claiming it.	/simulate
Road closure and re-routing	Plans must adapt when conditions change mid-event.	/dispatch, /simulate

Layer 5: Trust, ethics and demo control

Feature	Why it is needed	Where
Row Level Security by role	Vulnerable-person lists are sensitive, especially for women and children alone. RLS enforces privacy in the database itself.	Supabase
'Sensitive' flag hides exact location	Only the assigned volunteer and coordinators should see where a woman or child alone lives.	Supabase
Fairness panel and correction	Low-signal and low-income areas report less. Without a check they would be ranked lower for being quiet.	/quality
Insert-only audit log	Every override is traceable. Nobody can edit history.	/quality
Synthetic-data banner	Honesty: demo data is synthetic and weights are expert assumptions, not validated.	All pages
Demo control page	Makes the story repeatable and recoverable if something breaks on stage.	/admin/demo

4. What to build first, and what to cut

Group	Features	Reason
Never cut	Risk score with confidence; silence escalation; priority queue with reasons; offline field app; what-if toggles; role-based privacy; demo control page.	These are what the problem statement actually tests.
Build next	Optimizer with trade-off notes; override and audit; resident simulator; Telegram bot; registrar form; fairness panel.	They make the core story convincing.
Cut first (in this order)	Real SMS gateway and voice (keep the simulated transcript); shelter manager screen; intake review screen; viewer role; heatwave mode.	Lowest impact per hour of work.

Counts at a glance

Item	Count	Detail
Roles in database	5	Coordinator, volunteer, registrar, shelter manager, viewer
Non-login actors	1	Residents (Telegram, SMS, voice)
Routes (pages)	12	login, command, case, dispatch, simulate, field, resident-sim, register, shelter, quality, intake, admin/demo
Distinct screens / views	17	Includes case drawer, 4 field views, 3 quality tabs
Minimum viable pages	8	3 roles + residents
Edge Functions	6	tick, recompute, telegram-webhook, send-message, intake-parse, seed/reset-demo
Core tables	About 12	households, people, areas, hazard_state, checkins, cases, resources, assignments, reports, shelters, audit_log, demo_clock

Free-tier reminders: check Supabase's current free limits before relying on them, open the project the day before the demo because inactive free projects can pause, filter Realtime subscriptions by area, and keep a screen recording as a backup.